

Make stateful allocator propagation more consistent for operator+(basic_string)

Paper number P1165R1

Reply-to Tim Song <rs2740@gmail.com>

Audience LWG

Abstract

Allocator propagation for `basic_string`'s `operator+` is haphazard, inconsistent, and a source of implementation divergence. Let's make them consistent.

Revision history

- R1: Implements LEWG's [desired outcome](#) (leave rvalue overloads unchanged, lvalue overloads use SOCCC on lhs if possible and rhs otherwise). Note reservation in appendix.
- R0: Initial revision. Consistently default-constructs the allocator for the result.

Introduction

Let lhs and rhs be two `basic_string` lvalues with a `char value_type`. The following table shows the allocator used for the result for each of the 12 `operator+` overloads, in the working paper and three major implementations, as well as LEWG's preferred direction:

Expression	WP	libstdc++ (trunk)	libc++ (trunk)	MSVC STL (19.00.23506)	LEWG direction
lhs + rhs	SOCCC on SOCCC on lhs	SOCCC on lhs	lhs (no SOCCC)	default- constructed	SOCCC on lhs
lhs + std::move(rhs)	rhs	rhs	rhs	rhs	rhs
std::move(lhs) + rhs	lhs	lhs	lhs	lhs	lhs
std::move(lhs) + std::move(rhs)	lhs "or equivalently" rhs	lhs	lhs	lhs	lhs
lhs + "str"	default- constructed	SOCCC on lhs	lhs	default- constructed	SOCCC on lhs

Expression	WP	libstdc++ (trunk)	libc++ (trunk)	MSVC STL (19.00.23506)	LEWG direction
lhs + 'c'	default-constructed	SOCCC on lhs	lhs	default-constructed	SOCCC on lhs
std::move(lhs) + "str"	lhs	lhs	lhs	lhs	lhs
std::move(lhs) + 'c'	lhs	lhs	lhs	lhs	lhs
"str" + rhs	default-constructed	default-constructed	rhs	default-constructed	SOCCC on rhs
'c' + rhs	default-constructed	default-constructed	rhs	default-constructed	SOCCC on rhs
"str" + std::move(rhs)	rhs	rhs	rhs	rhs	rhs
'c' + std::move(rhs)	rhs	rhs	rhs	rhs	rhs

(SOCCC == `select_on_container_copy_construction`. The ~~struck through~~ text is proposed to be removed by P1148R0.)

Discussion

The [wording below](#) implements LEWG's [desired outcome](#). As the revision history makes clear, this is not the author's preferred direction. The [appendix](#) explains why.

Proposed wording

This wording is relative to [N4778](#).

Replace `[string.op+]` in its entirety with the following:

```
template<class charT, class traits, class Allocator>
    basic_string<charT, traits, Allocator>
        operator+(const basic_string<charT, traits, Allocator>& lhs,
                  const basic_string<charT, traits, Allocator>& rhs);

template<class charT, class traits, class Allocator>
    basic_string<charT, traits, Allocator>
        operator+(const basic_string<charT, traits, Allocator>& lhs, const charT* r
```

¹ *Effects*: Equivalent to:

```
basic_string<charT, traits, Allocator> r = lhs;
r.append(rhs);
return r;
```

```
template<class charT, class traits, class Allocator>
basic_string<charT, traits, Allocator>
operator+(basic_string<charT, traits, Allocator>&& lhs,
          const basic_string<charT, traits, Allocator>& rhs);
template<class charT, class traits, class Allocator>
basic_string<charT, traits, Allocator>
operator+(basic_string<charT, traits, Allocator>&& lhs, const charT* rhs);
```

² *Effects*: Equivalent to:

```
lhs.append(rhs);
return std::move(lhs);
```

```
template<class charT, class traits, class Allocator>
basic_string<charT, traits, Allocator>
operator+(basic_string<charT, traits, Allocator>&& lhs,
          basic_string<charT, traits, Allocator>&& rhs);
```

³ *Effects*: Equivalent to:

```
lhs.append(rhs);
return std::move(lhs);
```

except that both lhs and rhs are left in valid but unspecified states. [*Note*: If lhs and rhs have equal allocators, the implementation may move from either. — *end note*]

```
template<class charT, class traits, class Allocator>
basic_string<charT, traits, Allocator>
operator+(const basic_string<charT, traits, Allocator>& lhs,
          basic_string<charT, traits, Allocator>&& rhs);
template<class charT, class traits, class Allocator>
basic_string<charT, traits, Allocator>
operator+(const charT* lhs, basic_string<charT, traits, Allocator>&& rhs);
```

⁴ *Effects*: Equivalent to:

```
rhs.insert(0, lhs);  
return std::move(rhs);
```

```
template<class charT, class traits, class Allocator>  
basic_string<charT, traits, Allocator>  
operator+(const charT* lhs, const basic_string<charT, traits, Allocator>& rhs);
```



⁵ *Effects*: Equivalent to:

```
basic_string<charT, traits, Allocator> r = rhs;  
r.insert(0, lhs);  
return r;
```

```
template<class charT, class traits, class Allocator>  
basic_string<charT, traits, Allocator>  
operator+(charT lhs, const basic_string<charT, traits, Allocator>& rhs);
```

⁶ *Effects*: Equivalent to:

```
basic_string<charT, traits, Allocator> r = rhs;  
r.insert(r.begin(), lhs);  
return r;
```

```
template<class charT, class traits, class Allocator>  
basic_string<charT, traits, Allocator>  
operator+(charT lhs, basic_string<charT, traits, Allocator>&& rhs);
```

⁷ *Effects*: Equivalent to:

```
rhs.insert(rhs.begin(), lhs);  
return std::move(rhs);
```

```
template<class charT, class traits, class Allocator>  
basic_string<charT, traits, Allocator>  
operator+(const basic_string<charT, traits, Allocator>& lhs, charT rhs);
```

⁸ *Effects*: Equivalent to:

```

basic_string<charT, traits, Allocator> r = lhs;
r.push_back(rhs);
return r;

```

```

template<class charT, class traits, class Allocator>
basic_string<charT, traits, Allocator>
operator+(basic_string<charT, traits, Allocator>&& lhs, charT rhs);

```

⁹ *Effects*: Equivalent to:

```

lhs.push_back(rhs);
return std::move(lhs);

```

Appendix

The essence of the arguments below were made in a private discussion that was subsequently posted to the committee wiki prior to the San Diego meeting, to which LEWG has access. However, the remainder of this section *has not been seen by LEWG in this form and reflects the author's opinion only*.

Under LEWG's preferred approach, the result's allocator is sensitive to the value categories of the arguments, which may not be obvious from inspection of the code performing the concatenation:

```

using mystring = std::basic_string<char, char_traits<char>, my_allocator<char>;
struct person {
    const mystring& name() const; // maybe the name is stored directly
    mystring address() const; // but the full address is generated on request...
} p;

mystring foo = /* ... */, bar = /* ... */, baz = /* ... */;
foo + p.name() + ...; // uses SOCCC on foo's allocator
bar + p.address() + ...; // uses allocator of address()'s return value instead

```

and, because it breaks associativity with respect to allocator state, the result's allocator is also sensitive to simple refactoring:

```

baz + foo + bar; // SOCCC on baz's allocator

baz + (foo + bar); // SOCCC on foo's allocator instead

auto format_bar = [&](const mystring& b) { return foo + b; };
baz + format_bar(bar); // also SOCCC on foo's allocator

```

Additionally, it's not clear that unconditional propagation of allocator state for rvalues is desirable, since it may result in leakage of locally-scoped resources. `a + "str"` is also no longer merely a performance shortcut for `a + basic_string<...>("str")`, which may break the mental model of some people.

Nonetheless, it should be acknowledged that LEWG's preferred approach has benefits as well:

- it avoids breaking any existing code that depends on implementation convergence for the rvalue overloads;
- it permits using non-default-constructible allocators with `operator+`;
- it maintains the equivalence of `x += y` and `x = x + y`;
- it permits a hack to control the allocator to be used for the result, by supplying an rvalue `basic_string` with the desired allocator as the first operand;

It appeared unlikely to the author that there are much existing code relying on the rvalue overloads, considering the implementation divergence on the lvalue overloads. Under those circumstances, the author preferred a "break loudly and obviously" approach to the approach taken above, which in his view is prone to causing more subtle breakages.